

Students identification – João Martinho (106598) , Beatriz Coelho (107113)

# 1 Introduction

*Space Trash* is a distributed simulation game set in a universe threatened by the accumulation of space trash.

Players control trash-ships responsible for collecting trash and depositing it on a recycling planet.

A central server simulates the universe and applies physics rules in real time.

Clients allow user interaction, while a dashboard monitors global statistics.

The project explores distributed systems, networking, concurrency, and real-time simulation.

## 2 Part A

### 2.1 Implemented functionalities

Table 1: Implemented functionalities (PART A)

|                                             | Not implemented | With faults | Totally Correct     |
|---------------------------------------------|-----------------|-------------|---------------------|
| <b>General</b>                              |                 |             |                     |
| Planets storage                             |                 |             | x                   |
| Trash storage                               |                 |             | x                   |
| Trash ship storage                          |                 |             | x                   |
| Update of screen: planets, trash, ships     |                 |             | x                   |
| Universe configurations file                |                 |             | x                   |
| <b>Universe-simulator-main</b>              |                 |             |                     |
| Trash movement (physics rules)              |                 |             | x                   |
| New trash when trash hits planet            |                 |             | x                   |
| End of universe                             |                 |             | x                   |
| <b>Trash ship simulator universe-client</b> |                 |             |                     |
| Connect                                     |                 |             | x                   |
| reads cursor keys                           |                 |             | x                   |
| sends movement to the server                |                 |             | x                   |
| <b>Trash ship simulator universe-server</b> |                 |             |                     |
| Socket creation                             |                 |             | x                   |
| receives multiple client connections        |                 |             | x                   |
| Receives trash ships movement               |                 |             | x (local host only) |
| Move the trash ships                        |                 |             | x                   |

|                                     |  |   |   |
|-------------------------------------|--|---|---|
| Trash collected by trash ship       |  |   | x |
| Trash ship spill                    |  |   | x |
| Deposit trash in a recycling planet |  |   | x |
| <b>Misc</b>                         |  |   |   |
| Messages validation                 |  |   | x |
| Optimization of communication       |  |   | x |
| Code organization                   |  |   | x |
| Code reuse between components       |  |   | x |
| Functions return values validation  |  | x |   |

## 2.2 Description of faulty functionalities

In some functions we don't validate the return values.

## 2.3 Data types

In Part A, several data structures were defined to represent the universe, its physical behaviour, and the interaction with a single trash ship. These structures are declared in the structs.h file and are used throughout the universe simulator, server, and client components.

The planet\_structure struct stores all relevant information about each planet in the universe. It includes a short identifier (name), its position in the universe (x, y), the amount of recycled trash, the planet mass used for gravitational calculations, and a pointer to an SDL\_Color structure for graphical representation. Planets are stored in a fixed-size array, since their number is defined at initialization.

The vector struct is used to represent physical quantities related to movement, namely velocity and acceleration. It stores an amplitude and an angle, allowing movement to be represented in polar coordinates. This structure is used exclusively for trash movement in Part A.

The trash\_structure struct represents each piece of trash present in the universe. It stores its position, mass, acceleration and velocity vectors, graphical color, and state information indicating whether the trash has been collected (taken) and which ship collected it (ship\_name). Trash elements are stored in an array that is updated during the simulation.

The ship struct contains information related to the trash ship controlled by the client. It stores the ship position, identifier, authentication password, usage flag, and current trash cargo. In Part A, ship movement is not affected by physics rules.

Finally, the direction\_t enumeration defines the possible movement directions sent by the client to the server, while the respost enumeration is used to represent operation results in communication messages.

## 2.4 Code organization and Functions List

### 1. Universe-simulator-main

This component is responsible for simulating the universe without client interaction, including planets and trash behavior.

#### Files:

- **Universe-simulator.c**  
Main simulation loop. Initializes SDL, loads configurations, creates the universe, applies physics rules to trash, updates positions, renders the universe, and verifies end-of-universe conditions.
- **Universe-data.c / Universe-data.h**  
Functions to create, store, and manage planets and trash. Responsible for initializing data structures and providing access to universe data.
- **gravitation.c**  
Implements gravitational force calculations, acceleration, velocity, and position updates for trash elements.
- **display.c / display.h**  
Handles all graphical rendering using SDL, including drawing planets, trash, and background elements.
- **read\_conf.c / read\_conf.h**  
Reads configuration parameters from a configuration file (universe size, number of planets, trash limits, etc.) using libconfig.
- **structs.h**  
Defines all data structures used in the project, such as planets, trash, vectors, and ships.
- **conf.config**  
Defines the configuration variables use on the program.

### 2. Universe-server

The universe-server manages the universe state when trash ships are present and handles communication with clients.

#### Files:

- **Universe-server.c**  
Main server logic. Initializes the universe, accepts client connections, processes movement commands, updates ship positions, handles trash collection and deposition, and renders the universe.

- **Universe-data.c / Universe-data.h**  
Shared with the simulator. Manages planets, trash, and ships data structures on the server side.
- **display.c / display.h**  
Displays the universe state, including ships, planets, and trash, using SDL.
- **Comunnication.c / Comunnication.h**  
Implements ZeroMQ socket communication between server and clients. Responsible for sending and receiving messages and validating protocol correctness.
- **message\_comms.pb-c.c / message\_comms.pb-c.h**  
Auto-generated files from Protocol Buffers. Define message formats used for communication between server and client.
- **conf.config**  
Defines the configuration variables use on the program
- **message\_comms.proto**  
Defines the protocol messages.

### 3. Universe-client (Trash-ship client)

This component allows users to control a trash-ship and send commands to the server.

#### Files:

- **Player.c**  
Handles player logic, including ship identification and interaction with the server.
- **display\_client.c / display\_client.h**  
Displays the universe state received from the server, ensuring visual synchronization with the server display.
- **Comunnication.c / Comunnication.h**  
Sends movement commands to the server and receives universe updates via ZeroMQ sockets.
- **message\_comms.pb-c.c / message\_comms.pb-c.h**  
Used to encode and decode messages exchanged with the server.
- **conf.config**  
Defines the configuration variables use on the program
- **message\_comms.proto**  
Defines the protocol messages.

## 2.5 Communication

### 2.5.1 Sockets

In Part A, communication between the **universe-client** and the **universe-server** is implemented using ZeroMQ TCP sockets.

The server creates a socket and waits for client connections.

The client connects to the server and sends movement commands generated from keyboard input.

Messages are exchanged using a request–reply communication pattern.

Protocol Buffers are used to encode and decode the transmitted messages.

### 2.5.2 Transferred data

In Part A, the data exchanged between the **universe-client** and the **universe-server** is defined using **Protocol Buffers** messages, specified in the `message_comms.proto` file. These messages are transmitted through the ZeroMQ TCP sockets described in the previous section.

The **client\_connection\_req** message is sent by the client when it connects to the server. It contains a type field (default value "HELLO") used to identify the connection request and initiate the assignment of a trash ship.

The **server\_connection\_req** message is sent by the server in response to a connection request. It includes the connection result, the assigned ship name, and a password (pass) used to authenticate future messages from the client.

The **client\_movement\_req** message is sent by the client whenever a movement command is issued. This message contains the ship identifier (ch), the authentication password, and the movement direction, encoded using the `Direction_enum`.

The **server\_movement\_req** message is sent by the server after processing a movement request. It contains a `MoveResult` field indicating whether the movement was successful or blocked (e.g., collision with a wall).

The enumerations **Direction\_enum**, **MoveResult**, and **ConnectionResult** are used to standardize movement directions and operation results, ensuring consistent interpretation of messages exchanged over the sockets.

### 2.5.3 Error treatment

In Part A, several error treatment mechanisms were implemented to ensure reliable communication between the **universe-client** and the **universe-server**. All received messages are validated after decoding to confirm that the message type, ship identifier, and movement direction contain valid values.

Return values of socket send and receive operations are always checked to detect communication failures or disconnections. Message decoding using Protocol Buffers is also verified, and invalid or corrupted messages are discarded.

Additionally, authentication fields such as the ship password are validated by the server to prevent unauthorized commands. These measures help ensure robustness and prevent invalid or malicious client behavior.

## 3 Part B

### 3.1 Implemented functionalities

Table 2: Implemented functionalities (PART B)

|                                           | Not implemented | With faults | Totally Correct |
|-------------------------------------------|-----------------|-------------|-----------------|
| <b>General</b>                            |                 |             |                 |
| Planets storage                           |                 |             | x               |
| Trash storage                             |                 |             | x               |
| Trash ship storage                        |                 |             | x               |
| Update of screen: planets, trash, ships   |                 |             | x               |
| Universe configurations file              |                 |             | x               |
| <b>new-trash-ship-client</b>              |                 |             |                 |
| Connect                                   |                 |             | x               |
| Reads cursor keys                         |                 |             | x               |
| Sends movement to the server              |                 |             | x               |
| Update of screen: planets, trash, ships   |                 |             | x               |
| Detection of server failure               |                 | x           |                 |
| <b>new-universe-server</b>                |                 |             |                 |
| Initialization of planets and trash       |                 |             | x               |
| Trash movement (physics rules)            |                 |             | x               |
| New trash when trash hits planet          |                 |             | x               |
| Periodic trash generation                 |                 |             | x               |
| Interrupts trash generation with no ships |                 |             | x               |
| End of universe                           |                 |             | x               |
| Socket creation                           |                 |             | x               |
| Multiple client connections               |                 |             | x               |
| Receives trash ships movement             |                 |             | x               |
| Move the trash ships                      |                 |             | x               |
| Gravity and inertia of trash ships        |                 |             | x               |
| Trash collected by trash ship             |                 |             | x               |
| Trash ship spill                          |                 |             | x               |
| Trash ship capacity                       |                 |             | x               |
| Change of recycling planet                |                 |             | x               |
| Deposit trash in a recycling planet       |                 |             | x               |
| Update of screen: planets, trash, ships   |                 |             | x               |

|                                      |  |   |   |
|--------------------------------------|--|---|---|
| <b>new-dashboard</b>                 |  |   |   |
| Planet statistics update             |  |   | x |
| Ships statistics update              |  |   | x |
| Universe statistics                  |  |   | x |
| Different programming language       |  |   | x |
| <b>Misc</b>                          |  |   |   |
| Messages validation                  |  |   | x |
| Optimization of communication        |  |   | x |
| Code organization                    |  |   | x |
| Code reuse between client and server |  |   | x |
| Functions return values validation   |  | x |   |

## 3.2 Description of faulty functionalities

In some functions we don't validate the return values.

When the server closes abruptly, the client stay stuck waiting for the server response (but the server is not going to respond) because the client is running on timers to check if server connection still up and also on a different timer sends a message to the server so the server knows the client is still running. Because of this the client never closes.

## 3.3 Data types

In Part B, the server data structures were extended to support multiple trash-ships, physics-based movement, and concurrent execution. These structures are defined in the updated structs.h file.

The planet\_structure remains largely unchanged from Part A. It stores the planet identifier, position, accumulated recycled trash, mass used for gravitational calculations, and graphical color. Planets continue to be stored in a fixed-size array initialized at server startup.

The vector struct is reused and extended in Part B to represent both **velocity and acceleration** for trash and trash-ships. It stores an amplitude and an angle, allowing the server to apply gravity, inertia, friction, and thrust forces in a unified way.

The trash\_structure was updated to include a full physics model. Each trash element now stores both acceleration and velocity vectors, enabling continuous movement under gravitational influence. Additional fields identify whether the trash is currently collected and which ship is responsible for it. Trash elements are stored in a dynamically managed array that grows as new trash is generated.

The ship structure was significantly modified in Part B. Ships now include acceleration and velocity vectors, allowing movement to be governed by physics rules rather than direct position updates. Each ship stores its current position, identifier, activity status, authentication password, usage flag, and the amount of trash currently being transported. Ships are stored in a fixed-size array and managed concurrently by the server.

Global variables such as `player_radius` and `trash_radius` are used to standardize collision detection between ships, trash, and planets across the simulation.

Finally, the `direction_t` and `respost` enumerations are reused to encode movement commands and operation results exchanged between clients and the server.

### 3.4 Code organization and Functions List

The project is organized into several components, each responsible for a specific subsystem. In addition to the main components, there are shared and component-specific modules.

Main components:

- **new-universe-server** – central simulation server
- **new-trash-ship-client** – autonomous trash ship client
- **new-dashboard** – monitoring and visualization interface

Supporting directories:

- **common\_files** – shared modules
- **server\_only** – server-exclusive modules
- **trash\_ship\_only** – client-exclusive modules

- **3.4.1 common\_files**

- **Comunnication.h / Comunnication.c**

Implements all request/reply communication between clients and server using **ZeroMQ** and **Protocol Buffers**.

Main functions:

- `void *create_server_channel(void *ctx, const char *port)`
- `void *create_client_channel(const char *server_ip_addr, void *ctx, const char *port)`
- `void read_message(void *fd, char *message_type, char *c, direction_t *d, int *pass_out)`
- `void send_response(void *fd, respost rsp)`
- `void send_response_connection(void *fd, respost rsp, ship *player)`
- `int receive_response(void *fd)`
- `int receive_response_connection(void *fd, ship *ship)`
- `void send_connection_message(void *fd)`
- `void send_movement_message(void *fd, ship *player, direction_t d)`
- `void send_exit_message(void *fd, ship *player)`
- `void send_active_message(void *fd, ship *player)`

- **comms\_pub.c**

Implements PUB/SUB communication used to broadcast the universe state.

Main functions:

- `int send_world_state_pub(void *zmq_pub_socket, const planet_structure *planets, size_t n_planets, const trash_structure *trash, size_t n_trash, const ship *ships, size_t n_ships, int seq)`
- `int receive_world_state(void *sub_socket, planet_structure **planets_out, size_t *n_planets, trash_structure **trash_out, size_t *n_trash, ship **ships_out, size_t *n_ships, int *seq)`
- **message\_comms.pb-c.h / message\_comms.pb-c.c**  
Auto-generated files produced by **protobuf-c**.
- **read\_conf.h / read\_conf.c**  
Handles reading and parsing of configuration files.  
Main functions:
  - `SDL_Color getColor(const char *colorName)`
  - `config_t *open_conf_file(const char *file)`
  - `void read_configurations(...)`
  - `void read_network_config(...)`
- **structs.h**  
Defines shared data structures and enumerations, such as:
  - `planet_structure`
  - `trash_structure`
  - `ship`
  - `direction_t`
- **display.h / display.c**  
Shared SDL-based rendering utilities.  
Main functions:
  - `void SDL_SetRenderDrawColor_Direct_Color(SDL_Renderer *renderer, SDL_Color *color)`
  - `int initialize SDL_server(SDL_Window **window, SDL_Renderer **renderer, int window_size)`
  - `int initialize SDL_client(SDL_Window **window, SDL_Renderer **renderer, int window_size)`
  - `TTF_Font *initialize SDL_font(const char *font_path, int size)`
  - `void draw_planet(SDL_Renderer *renderer, planet_structure planet)`
  - `void draw_trash(SDL_Renderer *renderer, trash_structure trash)`
  - `void draw_ships(SDL_Renderer *renderer, ship *ships, int n_ships)`
  - `void render_all(SDL_Renderer *renderer, planet_structure *planets, int n_planets, trash_structure *trash, int n_trash, ship *ships, int n_ships)`
  - `void draw_server_close(SDL_Renderer *renderer, TTF_Font *font)`
  - `void draw_world_colapse_close(SDL_Renderer *renderer, TTF_Font *font)`
  - `void draw_server_collapsed(SDL_Renderer *renderer, TTF_Font *font)`
- **3.4.2 new-universe-server**  
This component maintains the authoritative universe state and executes the simulation.

- **new-universe-server.c**

Server entry point.

Main function:

- `int main(int argc, char *argv[])`

- **server\_only modules**

These modules are exclusive to the server component.

**threads\_server.h**

Defines shared server thread structures and global context variables.

**threads\_network.c**

Implements the server network thread.

Main function:

- `void *network_thread(void *arg)`

**threads\_physics.c**

Implements the physics and simulation thread.

Main function:

- `void *physics_thread(void *args)`

Responsibilities:

- Update positions and velocities.
- Apply gravitational effects and thrust.
- Detect collisions and universe collapse conditions.

**Universe-data.h / Universe-data.c**

Manages all universe entities.

Main functions:

- `planet_structure *create_planet(...)`
- `trash_structure create_trash(...)`
- `trash_structure *create_trash_init(...)`
- `ship *ship_init(...)`
- `int next_ship_use(...)`
- `void spiled_trash(...)`
- `void recicle_trash(...)`
- `int has_active_ship(...)`

**Player.h / Player.c**

Implements player and ship validation logic.

Main functions:

- `int check_info(...)`

- `int new_position(...)`
- `int same_color(...)`
- `int hit(...)`

#### **gravitation.h / gravitation.c**

Implements the gravitational physics model and thrust mechanics (server-exclusive).

Main functions:

- `void set_universe_size(int size)`
- `vector make_vector(float x, float y)`
- `vector add_vectors(vector v1, vector v2)`
- `void correct_position(trash_structure *t)`
- `void correct_position_f(ship *s)`
- `void new_trash_acceleration(...)`
- `void new_ship_acceleration(...)`
- `void new_trash_velocity(...)`
- `void new_ship_velocity(...)`
- `void new_trash_position(...)`
- `void new_ship_position(...)`
- `void new_ship_thrust(ship *s, direction_t d)`

#### • **3.4.3 new-trash-ship-client**

Implements an autonomous client that controls a trash-collecting ship.

#### • **new-trash-ship-client.c**

Client entry point.

Main function:

- `int main(int argc, char *argv[])`

#### • **trash\_ship\_only modules**

##### **thread.h / thread.c**

Implements the subscription thread used to receive the universe state.

Main functions:

- `void free_world(planet_structure *planets, size_t n_planets, trash_structure *trash, size_t n_trash, ship *ships)`
- `void *sub_thread_func(void *arg)`

#### • **3.4.4 new-dashboard**

The dashboard component provides real-time monitoring and visualization of the universe state and is implemented in **Python**.

#### • **new-dashboard.py**

Main function:

- `def read_conf_keys(filename): ...`

- **message\_comms\_pb2.py**  
Auto-generated Python file from message\_comms.proto.

## 3.5 Concurrency

### 3.5.1 Threads

#### **new-universe-server**

The **new-universe-server** component is a multi-threaded application responsible for managing the universe simulation and coordinating all connected clients. It implements the following threads:

#### **Network Thread**

**File:** server\_only/threads\_network.c

**Function:** void \*network\_thread(void \*arg)

#### **Objective:**

The network thread is responsible for handling part of communication between the server and connected clients.

#### **Functioning:**

- Listens for incoming client messages (connection requests, movement commands, activity signals, and exit requests).
- Parses and validates received messages.
- Updates shared server data structures according to client actions.
- Send appropriate responses back to clients.
- Synchronizes access to shared universe data using mutexes to avoid race conditions.

This thread ensures reliable and synchronized interaction between external clients and the internal simulation.

#### **Physics Thread**

**File:** server\_only/threads\_physics.c

**Function:** void \*physics\_thread(void \*args)

#### **Objective:**

The physics thread is responsible for continuously updating the simulation state of the universe.

#### **Functioning:**

- Executes a periodic simulation loop.
- Applies gravitational forces and thrust using the gravitation module.

- Updates positions and velocities of ships and trash.
- Detects collisions between ships, planets, and trash.
- Handles recycling events and universe collapse conditions.
- Updates the shared universe state that is later broadcast to clients and dashboards.

This thread guarantees a consistent and deterministic evolution of the universe over time.

## **Main Thread**

**File:** new-universe-server.c

**Function:** int main(int argc, char \*argv[])

### **Objective:**

The goal is to coordinate the SDL and send universe update.

### **Functioning:**

- Establishes the server channel.
- Creates and manages the physics and communication thread.
- SDL processing.

## **new-trash-ship-client**

The **new-trash-ship-client** component is also multi-threaded and separates communication from main execution logic.

## **Subscription Thread**

**File:** trash\_ship\_only/thread.c

**Function:** void \*sub\_thread\_func(void \*arg)

### **Objective:**

The subscription thread is responsible for receiving the current universe state from the server.

### **Functioning:**

- Continuously receives serialized universe state messages.
- Deserializes the received data (planets, trash, and ships).
- Updates the local representation of the universe used by the client logic.

This thread allows the client to maintain an up-to-date view of the universe without blocking its main execution flow.

## **Main Thread**

**File:** new-trash-ship-client.c

**Function:** int main(int argc, char \*argv[])

**Objective:**

The goal is the SDL render and cursor management.

**Functioning:**

- Establishes the initial connection to the server.
- SDL processing.
- Sends movement, activity, and exit commands to the server.

This separation allows the client to react to real-time universe updates while maintaining autonomous control logic.

### 3.5.2 Shared variables

The server component shares a global universe state between the **network thread** and the **physics thread**. All shared data is protected using a mutex to ensure thread safety.

The variables planets, trash, and ships store the current universe entities and are accessed by both threads for simulation updates and for sending the world state to clients.

Configuration variables such as n\_planets, initial\_trash, max\_trash, window\_size, planet\_radius, trash\_radius, and player\_radius define the universe dimensions and collision parameters and are mainly accessed by the physics thread.

Runtime control variables include ship\_active, which indicates active ships, current\_trash, which tracks the amount of trash in the universe, and was\_colliding, used to prevent repeated collision events. These variables are accessed by the physics thread and, when necessary, by the network thread.

The flags quit and universe\_collapsed control server shutdown and universe collapse conditions and are accessed by both threads.

A global mutex, world\_mutex, is used by both the network and physics threads to synchronize access to the shared universe state.

Network configuration variables port and port\_sub define the communication ports and are accessed by the network thread.

Shared color definitions (red, blue, green, bg0, black) are used by both threads for visualization and logic consistency.

The client component shares world state data between the **main thread** and the **subscription thread**.

The pointers `planets`, `trash`, and `ships`, along with their respective counters `n_planets`, `n_trash`, and `n_ships`, store the most recent universe state received from the server. These variables are updated by the subscription thread and read by the main thread for decision-making.

Control flags `running` and `has_world` are used to manage the lifecycle of the subscription thread and to signal when valid world data is available. The variable `world_update` records the latest update event.

Access to all shared world data is synchronized using the `world_mutex`, which is shared between both threads. The subscription socket `fd_sub` is exclusively used by the subscription thread.

Event codes defined in `world_event_code` are shared constants used to signal world updates and server state changes between threads.

### 3.5.3 Synchronization

#### **new-universe-server**

The server component uses a single mutex to protect the global universe state shared between the **network thread**, **main thread** and the **physics thread**.

- `pthread_mutex_t world_mutex`

#### **Type of synchronization:**

Mutual exclusion (mutex locking).

#### **Guarded variables:**

- `planets`
- `trash`
- `ships`
- `ship_active`
- `current_trash`
- `was_colliding`
- `universe_collapsed`
- `quit`

#### **Accessed in functions:**

- `network_thread()`
- `physics_thread()`
- `main()`

#### **new-trash-ship-client**

The client component also uses a mutex to synchronize access to the local universe representation shared between threads.

- `pthread_mutex_t world_mutex`

#### **Type of synchronization:**

Mutual exclusion (mutex locking).

#### **Guarded variables:**

- `planets`
- `trash`
- `ships`
- `n_planets`
- `n_trash`

- n\_ships
- has\_world

**Accessed by threads:**

- Subscription thread
- Main thread

**Accessed in functions:**

- sub\_thread\_func()
- main()

## 3.6 Communication

### 3.6.1 Sockets

In Part B, a new socket mechanism was introduced to support real-time distribution of the universe state to multiple components. This mechanism is implemented in the common communication module and is based on **ZeroMQ PUB/SUB sockets**.

A **PUB socket** is created on the server side using the function `create_pub_channel()`. This socket is bound to a dedicated TCP port and is responsible for broadcasting the complete universe state, including planets, trash, and ships. The universe state is serialized using **Protocol Buffers** and sent as a multipart message composed of a topic frame and a binary data frame. This socket is mainly used by the server to periodically publish simulation updates.

A **SUB socket** is created on the client side using the function `create_sub_channel()`. This socket connects to the server's PUB socket and subscribes to all available topics. A receive timeout is configured to avoid blocking indefinitely. The client receives and deserializes the universe state using the function `receive_world_state()`, updating its local data structures accordingly. Sequence numbers embedded in the messages are used to discard outdated packets and to detect special events such as server shutdown or universe collapse.

### 3.6.2 Transferred data

In Part B, a new **World State message** was introduced to represent the complete state of the universe. This message aggregates all simulation entities, including planets, trash objects, and ships, and is transmitted periodically using a PUB/SUB communication model.

The new WorldState message includes:

- A **sequence number** used to discard outdated packets and detect special events.
- An array of **planets**, each containing position, mass, trash count, name, and color.
- An array of **trash objects**, including position, mass, color, ownership state, and associated ship (if any).
- An array of **ships**, including position, activity status, name, and collected trash.

All transferred data is serialized using **Protocol Buffers**, ensuring compact and structured message representation across components written in different languages.

### 3.6.3 Error treatment

The PUB/SUB communication implemented in `comms_pub.c` includes several error handling mechanisms to ensure reliable transmission of the universe state.

During socket creation, the return values of `zmq_socket`, `zmq_bind`, and `zmq_connect` are verified. If any of these operations fail, the socket is closed and the function returns an error, preventing invalid communication channels from being used.

All send and receive operations validate their return values. Calls to `zmq_send` and `zmq_recv` check whether the expected number of bytes was transmitted or received. Messages with invalid or incomplete data are discarded and not processed further.

A receive timeout is configured on the subscriber socket using `ZMQ_RCVTIMEO`, ensuring that the receiver does not block indefinitely if the publisher becomes unavailable or communication is interrupted.

After receiving data, all messages are validated at the application level. The topic frame is checked to ensure a valid multipart message, and the deserialization process verifies the integrity of the received `WorldState` message. If deserialization fails, the message is discarded.

Sequence numbers embedded in the `WorldState` message are used to discard outdated packets and to detect special control conditions, such as server shutdown or universe collapse. Messages that do not satisfy these conditions are ignored.

## 4 Conclusion

This project allowed the practical application of concepts related to concurrency, inter-process communication, and data serialization. The main accomplishment was the successful extension of the system to support real-time dissemination of the universe state using a PUB/SUB communication model.

The use of ZeroMQ and Protocol Buffers enabled efficient and scalable communication between the server, clients, and monitoring components. Proper synchronization using mutexes was essential to maintain data consistency across multiple threads.

The main difficulties encountered were related to thread coordination, data validation, and debugging communication issues. Overcoming these challenges contributed to a better understanding of distributed systems and multi-threaded application design.

Overall, the project achieved its objectives and provided valuable experience in building robust and extensible concurrent systems.